ChatGPT said:

For your **AI Course-II AIML Practical Lab – Category 4**, here are the four Python programs for Uninformed Search Techniques.

I'll use simple graph examples so that the programs are easy to understand, execute in VS Code, and write in your practical record.

Category 4 – Uninformed Search Techniques

No.	Algorithm	Example use case
1	Breadth First Search (BFS)	Shortest path in an unweighted graph
2	Depth First Search (DFS)	Graph traversal
3	Depth Limited Search (DLS)	Searching with a depth limit
4	Bidirectional Search	Finding a path between two nodes

---

1. Breadth First Search (BFS)

Aim: To implement Breadth First Search to traverse a graph level by level.

Python Program:

Python

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

def bfs(start, goal):
    queue = deque([[start]])
    visited = set()

    while queue:
        path = queue.popleft()
        node = path[-1]

        if node == goal:
            print("Path Found:", path)
            return

        if node not in visited:
            visited.add(node)

            for neighbor in graph[node]:
                new_path = path + [neighbor]
```

Output:

Plain text

```
Path Found: ['A', 'C', 'G']
```

Explanation:

1. Start from node A.
2. Insert A into the queue.
3. Remove the first node from the queue.

4. Visit its unvisited neighboring nodes.
5. Add the neighboring nodes to the queue.
6. Repeat until the goal node is found.

Key point: BFS uses a FIFO (First In First Out) queue.

---

2. Depth First Search (DFS)

Aim: To implement Depth First Search to traverse a graph by exploring one branch completely before backtracking.

Python Program:

Python

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

visited = set()

def dfs(node, goal):

    if node == goal:
        print("Goal Found:", node)
        return True

    visited.add(node)
    print("Visited:", node)

    for neighbor in graph[node]:

        if neighbor not in visited:
            if dfs(neighbor, goal):
                return True

    return False
```

Output:

Plain text

```
Visited: A
Visited: B
Visited: D
Visited: E
Visited: C
Visited: F
Goal Found: G
```

Explanation:

1. Start from node A.
2. Mark A as visited.
3. Visit the first unvisited neighboring node.
4. Continue deeper into the graph.
5. If no unvisited neighbor exists, backtrack.
6. Stop when the goal is found.

Key point: DFS uses a stack or recursion (LIFO).

3. Depth Limited Search (DLS)

Aim: To implement Depth Limited Search by restricting the maximum search depth.

Python Program:

Python

```python
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

def dls(node, goal, limit, depth=0):

    print("Visiting:", node)

    if node == goal:
        print("Goal Found:", node)
        return True

    if depth == limit:
        return False

    for neighbor in graph[node]:

        if dls(neighbor, goal, limit, depth + 1):
            return True

    return False
```

Output:

Plain text

```
Visiting: A
Visiting: B
Visiting: D
Visiting: E
Visiting: C
Visiting: F
Visiting: G
Goal Found: G
```

Explanation:

1. Start from node A at depth 0.
2. Visit neighboring nodes.
3. Increase depth by 1 for every move.
4. Stop expanding a node when the depth limit is reached.
5. If the goal is found, display the result.

Key point: DLS is DFS with a fixed depth limit.

4. Bidirectional Search

Aim: To implement Bidirectional Search by searching simultaneously from the initial node and the goal node.

Python Program:

Python

```python
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F', 'G'],
    'D': ['B'],
    'E': ['B'],
    'F': ['C'],
    'G': ['C']
}

def bidirectional_search(start, goal):

    q1 = deque([[start]])
    q2 = deque([[goal]])

    visited1 = {start: [start]}
    visited2 = {goal: [goal]}

    while q1 and q2:

        path1 = q1.popleft()
        node1 = path1[-1]

        for neighbor in graph[node1]:

            if neighbor not in visited1:
                new_path = path1 + [neighbor]
```

Output:

Plain text

```
Path Found: ['A', 'C', 'G']
```

Explanation:

1. Start BFS from the initial node A.
2. Start another BFS from the goal node G.
3. Expand nodes from both directions.
4. Check whether both searches have reached a common node.
5. If they meet, combine both paths.
6. Display the final path.

Key point: Bidirectional Search performs two simultaneous searches.

Important viva questions

Question	Answer
What is uninformed search?	Search without heuristic information.
Which data structure is used in BFS?	Queue.
Which data structure is used in DFS?	Stack.
What is FIFO?	First In First Out.
What is LIFO?	Last In First Out.
Which algorithm finds the shortest path in an unweighted graph?	BFS.
What is backtracking?	Returning to a previous node when no further path is available.

What is DLS?	DFS with a predefined depth limit.
What is the main advantage of bidirectional search?	It can reduce the search space.
What happens if the DLS limit is too small?	The goal may not be found.

Comparison of four algorithms

Feature	BFS	DFS	DLS	Bidirectional
Data structure	Queue	Stack	Stack	Two queues
Search style	Level-wise	Deep-first	Limited depth	Both directions
Complete	Yes*	Not always	Not always	Yes*
Optimal	Yes*	No	No	Yes*

*Assuming a finite graph for BFS and bidirectional search, and equal edge costs for optimality.

My recommendation for your lab exam: Learn BFS first, then DFS, DLS and Bidirectional Search. All four programs use the same graph representation, so understanding BFS will make the remaining three much easier.